

Τελική Άσκηση στον Προγραμματισμό Υπολογιστών με C++

Επαναληπτική εξεταστική περίοδος Φεβρουαρίου 2024

Περιγραφή

Ο σκοπός της παρούσας άσκησης είναι να υλοποιήσετε κατάλληλα τις συναρτήσεις και κλάσεις που απαιτούνται για τη δημιουργία μιας απλής βάσης δεδομένων με στοιχεία φοιτητών. Η εγγραφή κάθε φοιτητή έχει τη μορφή (αρχείο student.h):

```
struct StudentData
{
    char m_name[20];
    char m_surname[20];
    int m_id;
};
```

Το πεδίο m_id είναι ο αριθμός μητρώου και αποτελεί μοναδικό αναγνωριστικό (κλειδί) για τη βάση μας. Η λειτουργικότητα της άσκησης αναπτύσσεται σταδιακά, μέσα από 5 «στάδια» που φαίνονται στο κεντρικό αρχείο κώδικα (exam.cpp) και ο σκοπός των οποίων εξηγείται παρακάτω:

Στάδιο 1. Ανάγνωση μιας εγγραφής από το αρχείο της βάσης (βαθμός 1.0).

Δημιουργία μιας συνάρτησης ανάγνωσης μιας εγγραφής τύπου StudentData από το δυαδικό raw αρχείο που είναι αποθηκευμένες σειριακά όλες οι εγγραφές. Στο αρχείο αυτό δεν υπονοείται κάποια ταξινόμηση και τα στοιχεία έχουν αποθηκευτεί το ένα μετά το άλλο σε πλειάδες μεγέθους sizeof(StudentData). Η συνάρτηση διαβάζει την επόμενη διαθέσιμη εγγραφή, αν υπάρχει.

Στάδιο 2. Αναζήτηση εγγραφής στο αρχείο της βάσης (βαθμός 1.0).

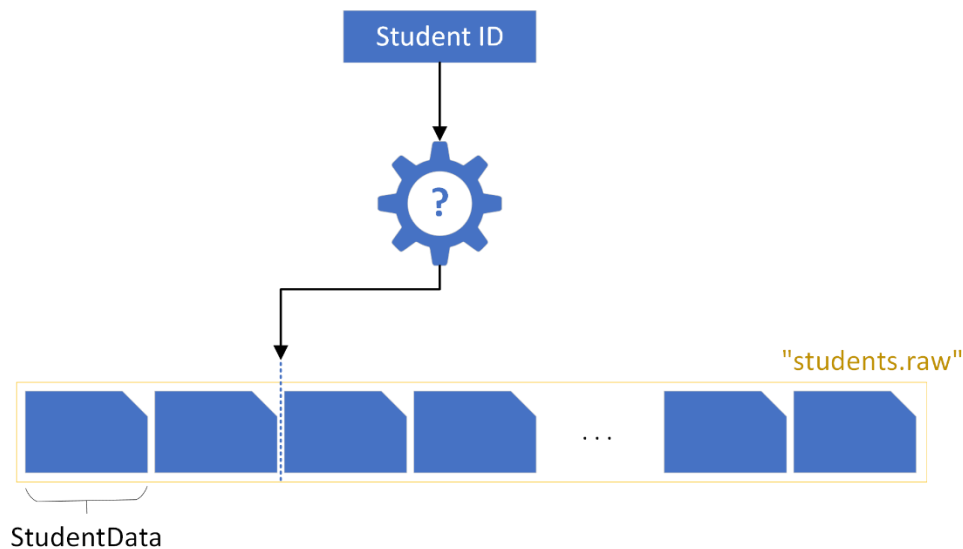
Δημιουργία μιας συνάρτησης για την αναζήτηση μιας εγγραφής (αν υπάρχει) στο δυαδικό raw αρχείο που είναι αποθηκευμένες σειριακά όλες οι εγγραφές, με βάση τον αριθμό μητρώου του φοιτητή (m_id στη δομή StudentData). Δεδομένου ότι δεν υπονοείται κάποια ταξινόμηση, η αναζήτηση είναι σειριακή.

Στάδιο 3. Δημιουργία κλάσης για την αναπαράσταση της βάσης (βαθμός 1.0).

Ενσωμάτωση της λειτουργικότητας αναζήτησης σε σειριακό raw αρχείο εγγραφών σε αντικειμενοστραφή αναπαράσταση. Δηλώστε και υλοποιήστε μια κλάση StudentDataBase με την ίδια λειτουργικότητα του σταδίου 2.

Στάδιο 4. Υλοποίηση index πάνω στα αποθηκευμένα δεδομένα (βαθμός 1.5).

Η σειριακή αναζήτηση σε ένα μεγάλο αρχείο είναι αργή. Να υλοποιηθεί μηχανισμός στη μνήμη (όχι δηλαδή σε αρχείο στο δίσκο, που θα ήταν πιο δύσκολο) για την αντιστοίχιση αριθμού μητρώου (m_id) με τη θέση στο αρχείο στην οποία ξεκινάνε τα δεδομένα της σχετικής εγγραφής (βλ. εικόνα). Να γίνεται πλέον η αναζήτηση εγγραφών με αυτό τον μηχανισμό αντί για τη σειριακή αναζήτηση που υλοποιείται στα στάδια 2 και 3. Η αποδοτική υλοποίηση (με χρήση των κατάλληλων δομών STL) συνεκτιμάται στο βαθμό.



Στάδιο 5. Υλοποίηση template κλάσης που να υλοποιεί μια απλή βάση δεδομένων με ή χωρίς index (-0.5) για δεδομένα τυχαίου τύπου (βαθμός 1.5).

Σε αυτό το στάδιο δε θεωρείται γνωστός από πριν ούτε ο τύπος της εγγραφής που αποθηκεύεται στο αρχείο, ούτε ο τύπος του κλειδιού αναζήτησης. Το template θα πρέπει να έχει 2 ορίσματα:

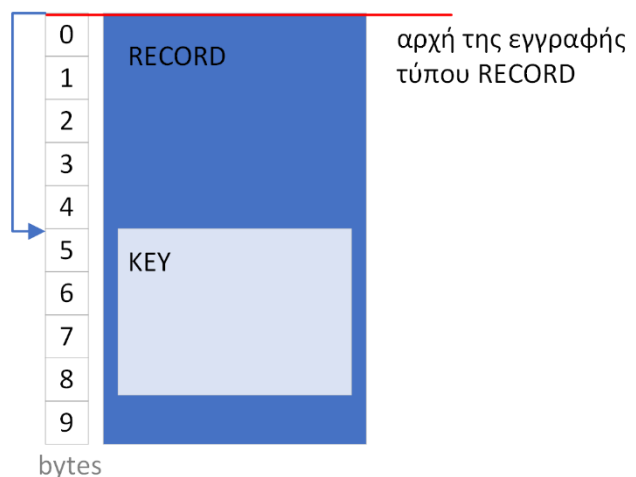
- RECORD: Ο αφηρημένος τύπος της εγγραφής
- KEY: Ο αφηρημένος τύπος του κλειδιού

Η μόνη προϋπόθεση που γνωρίζουμε για το κλειδί είναι ότι μπορεί να ελεγχθεί η ισοτιμία του (==) και η διάταξή του (<) σε σχέση με άλλα κλειδιά. Μπορεί π.χ. να είναι std::string, float, int, size_t κλπ.

Επειδή δε γνωρίζουμε τη μορφή των εγγραφών που αποθηκεύονται στη βάση μας, για να μπορέσουμε να εντοπίσουμε το κλειδί σε ένα RECORD, θα πρέπει να δώσουμε εμείς την απόσταση σε bytes από την αρχή του τύπου RECORD όπου υπάρχει μέσα στην εγγραφή το κλειδί μας. Αυτό κάνουν οι γραμμές στον κώδικα του exam.cpp :

```
const int offset = (char*)&student.m_id - (char*)&student;
...
```

```
db.setKeyOffset(offset);
```



Ζητούμενα

Ζητούμενα της άσκησης είναι:

A) Το αρχείο db.h με τον πηγαίο κώδικα δήλωσης και υλοποίησης των συναρτήσεων και κλάσεων που απαιτούνται στα στάδια 1 – 5.

B) Το κείμενο της εξόδου του εκτελέσιμου προγράμματος αντιγραμμένο σε ένα αρχείο κειμένου (.txt). Ακόμα και αν δεν έχετε υλοποιήσει μέρος της λειτουργικότητας ή έχετε σφάλματα, αυτό είναι υποχρεωτικό παραδοτέο.

Και τα 2 ζητούμενα είναι υποχρεωτικά για να βαθμολογηθεί η εργασία σας. Ανεβάζετε τα 2 αρχεία στα αντίστοιχα πεδία στη φόρμα της εργασίας στο eclass.

Άλλες Οδηγίες

Ο κώδικάς που σας δίνεται αποτελείται από 3 αρχεία header και το κύριο αρχείο cpp που περιλαμβάνει τη main. Αναλυτικά:

- **main.cpp:** ο κώδικας για το κύριο πρόγραμμα και τις βασικές συναρτήσεις ελέγχου του κώδικά σας Stage1() – Stage5(). Μέσα από αυτό τον κώδικα δημιουργούνται αντικείμενα και καλούνται μέθοδοι και ελεύθερες συναρτήσεις που πρέπει εσείς να υλοποιήσετε.
- **Timer.h:** Περιλαμβάνει τη δήλωση και υλοποίηση για μια κλάση μέτρησης χρόνου που αποτιμά την επίδοση της υλοποίησής σας και τυπώνει τα αποτελέσματα στην έξοδο.
- **Student.h:** Περιλαμβάνει τη βασική δήλωση της δομής StudentData.
- **Init.h:** Περιλαμβάνει κώδικα για την αρχικοποίηση των δεδομένων στο δίσκο κατά την εκτέλεση του προγράμματός σας.

ο κώδικας πρέπει να μπορεί να μεταγλωττιστεί και να φτιαχτεί ένα εκτελέσιμο χωρίς εσείς να χρειάζεται να προσθέσετε κώδικα (δεν παίρνετε κάποιο βαθμό γι αυτό). Κατά την εκτέλεση, θα πρέπει να εμφανίζεται:

```
Created raw database file "students.raw".
```

Για να ενεργοποιηθούν οι διάφορες συναρτήσεις που καλούν το δικό σας κώδικα, πρέπει να βγάλετε από σχολία τις γραμμές 9 – 13.

Προσοχή: **ΔΕΝ ΠΡΕΠΕΙ ΝΑ ΜΕΤΑΒΑΛΕΤΕ ΚΑΝΕΝΑ ΑΡΧΕΙΟ ΑΠΟ ΑΥΤΑ ΠΟΥ ΣΑΣ ΔΙΝΟΝΤΑΙ, πέρα από το σχολιασμό / αφαίρεση σχολίων για τη σταδιακή ανάπτυξη του κώδικά σας στο exam.cpp.**

Για την ανάπτυξη του κώδικα μπορείτε προφανώς να χρησιμοποιήσετε οποιονδήποτε compiler C++ (έκδοση γλώσσας 11 και μετά), IDE και λειτουργικό σύστημα. Ο κώδικας έχει ελεγχθεί όμως σε VisualStudio 2022 σε Windows10/11 (x64). Αν χρησιμοποιείτε άλλον compiler, ενδέχεται να χρειαστεί να προσθέσετε Include files που πιθανά να απουσιάζουν από την τρέχουσα μορφή του δεδομένου κώδικα.

Ενδεικτική εκτέλεση (τα ακριβή αποτελέσματα ενδέχεται να διαφέρουν):

Created raw database file "students.raw".

STAGE1: Read the first record from the raw database file.
[PASS] First record in raw database is (5001610, Giorgos, Dalaras)

STAGE2: Find a single record in the raw database file.
[PASS] Found student with id 5000001: Voula Mpoula

STAGE3: Object-oriented, non-indexed database access / Multiple queries (slow).
[PASS] Throughput: 0.00150026 queries/sec.

STAGE4: Indexed database access / Multiple queries (fast).
[PASS] Throughput: 934579 queries/sec.

STAGE5: Database template / Generalized representation.
[PASS] Found student with id 5000002: Peppa the Pig